



UniCathedral

CENTRO UNIVERSITÁRIO CATHEDRAL
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

NOME DO ALUNO

PROJETO DE SOFTWARE
SISTEMA DE CONTROLE BANCÁRIO

BARRA DO GARÇAS – MT
2026

NOME DO ALUNO

PROJETO DE SOFTWARE
SISTEMA DE CONTROLE BANCÁRIO

Trabalho apresentado à disciplina de Engenharia de Software, do curso de Análise e Desenvolvimento de Sistemas, do Centro Universitário Cathedral, como parte dos requisitos para aprovação na disciplina.

Orientador: Prof. Me. Josiel S. Moura

BARRA DO GARÇAS – MT
2026

Sumário

INTRODUÇÃO.....	3
1. DESCRIÇÃO DO SISTEMA.....	4
2. REQUISITOS FUNCIONAIS.....	5
3. REQUISITOS NÃO FUNCIONAIS	6
4. DIAGRAMA DE CASOS DE USO.....	6
5. DIAGRAMAS DE ATIVIDADE	7
6. DEFINIÇÃO DA IMPLEMENTAÇÃO.....	12
CONSIDERAÇÕES FINAIS	14
REFERÊNCIAS BIBLIOGRÁFICAS	15

INTRODUÇÃO

No cenário contemporâneo, a transformação digital redefiniu a forma como a sociedade interage com as instituições financeiras. O surgimento de *fintechs* e a consolidação do *internet banking* elevaram o nível de exigência dos usuários por sistemas que sejam, simultaneamente, ágeis, seguros e intuitivos. (Informação externa ao material fornecido). Nesse contexto, a Engenharia de Software desempenha um papel crucial ao fornecer metodologias e padrões de modelagem que garantem que as regras de negócio complexas do setor bancário sejam traduzidas fielmente em sistemas computacionais robustos e escaláveis.

Para alcançar esse nível de precisão, a utilização da linguagem UML (*Unified Modeling Language*) é fundamental, pois permite a visualização clara da arquitetura e do comportamento do software antes mesmo do início da codificação. As fontes destacam que a definição de uma "fronteira de sistema" é o primeiro passo para identificar quais funcionalidades são internas e como os atores externos interagem com elas. Esta abordagem estruturada minimiza falhas de comunicação entre desenvolvedores e clientes, garantindo que o produto final atenda aos requisitos levantados.

Este projeto detalha a modelagem de um **Sistema de Controle Bancário**, uma ferramenta essencial para a gestão de operações financeiras cotidianas. O sistema permite que clientes realizem depósitos, saques e consultas de saldos ou extratos, preferencialmente via caixas eletrônicos, enquanto operações mais complexas, como a abertura e o encerramento de contas, ocorrem sob a supervisão de funcionários. A modelagem abrange diferentes modalidades de contas — **comum, especial (com limite de crédito) e poupança (com rendimentos)** — adaptando-se às diversas necessidades do público bancário.

Por fim, o presente trabalho é desenvolvido como requisito para a disciplina de Engenharia de Software do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário Cathedral. O objetivo principal é aplicar os conceitos de diagramas de casos de uso e de atividades para estruturar um sistema que respeite dependências críticas, como a obrigatoriedade de um depósito inicial na abertura de conta ou a liquidação de saldos pendentes antes de um encerramento.

1. DESCRIÇÃO DO SISTEMA

O Sistema de Controle Bancário foi projetado para atuar como uma ferramenta centralizada na gestão de operações financeiras, automatizando o fluxo de trabalho entre clientes e a instituição. O software opera em dois níveis de acesso principais: o primeiro voltado ao autoatendimento, onde o cliente realiza transações rápidas em terminais eletrônicos, e o segundo voltado ao ambiente administrativo, onde funcionários conduzem procedimentos de registro e manutenção. A arquitetura do sistema segue uma lógica procedural, focada na execução sequencial de funções que garantem a integridade das operações de saque, depósito e consulta, tratando cada transação como um conjunto de dados processados conforme regras de negócio pré-definidas.

O processo de abertura de conta é realizado exclusivamente por um funcionário e segue um fluxo lógico rigoroso para garantir a validade dos dados. Ao iniciar este procedimento, o sistema verifica as informações cadastrais do cliente; caso o usuário não possua um registro prévio ou necessite de atualizações, a função de manutenção de cadastro é acionada antes que a conta seja gerada. Uma regra fundamental estabelecida para esta etapa é a obrigatoriedade de um depósito inicial, o que significa que o sistema não permite a conclusão da abertura de uma conta sem que haja uma movimentação financeira de entrada registrada simultaneamente. Este modelo unificado de abertura simplifica a entrada de dados, tratando as particularidades de cada conta como parâmetros dentro da mesma função principal.

No cotidiano operacional, os clientes interagem com o sistema para gerenciar seus recursos de forma autônoma através de funções de movimentação e consulta. Para qualquer operação de retirada ou consulta, o sistema exige a validação da senha cadastrada, assegurando que apenas usuários autorizados acessem as informações. Ao realizar um depósito, o sistema processa o valor e atualiza o saldo disponível, enquanto no saque é realizada uma verificação prévia para garantir que a operação não exceda os limites permitidos. Todas essas ações geram registros históricos de movimentação, que alimentam as funções de emissão de saldo e extrato, permitindo que o cliente acompanhe detalhadamente suas atividades financeiras ao longo do tempo.

O encerramento de uma conta é um procedimento crítico que exige a liquidação total de valores pendentes para ser concluído. Antes de finalizar o registro, o sistema executa obrigatoriamente uma consulta de saldo para determinar se há necessidade de realizar um último saque, caso existam fundos, ou um depósito, caso haja débitos a serem

quitados. Uma vez que o saldo esteja zerado, o sistema altera o status da conta para inativo. É um requisito de segurança e integridade que o sistema jamais apague definitivamente os registros de contas ou de clientes do banco de dados, mantendo todas as informações armazenadas para fins de auditoria e histórico bancário, mesmo após o término do vínculo com a instituição.

2. REQUISITOS FUNCIONAIS

Requisitos funcionais (RF), para Valente, definem o que um sistema deve fazer, descrevendo comportamentos, ações específicas, processamento de dados e interações com o usuário. Eles focam nas funcionalidades necessárias para atender às necessidades do negócio e dos usuários,

De acordo com as funcionalidades descritas, os principais requisitos funcionais são:

RF01: Abrir Conta: Permitir a criação de contas comuns, especiais ou poupança.

RF02: Encerrar Conta: Permitir o fechamento de contas, condicionado à verificação de saldo.

RF03: Realizar Depósito: Registrar a entrada de valores em uma conta.

RF04: Realizar Saque: Permitir a retirada de valores, respeitando o saldo ou limite disponível.

RF05: Emitir Saldo: Exibir o montante disponível na conta do cliente.

RF06: Emitir Extrato: Listar a movimentação da conta em um período solicitado.

RF07: Manter Clientes: Registrar novos clientes ou alterar dados de clientes já cadastrados.

3. REQUISITOS NÃO FUNCIONAIS

Segundo Miranda, requisitos não funcionais (RNFs) definem como um sistema deve operar, estabelecendo atributos de qualidade como desempenho, segurança, usabilidade e confiabilidade. Ao contrário dos funcionais (que dizem o que o sistema faz), os RNFs são restrições técnicas que garantem a eficiência e a experiência do usuário.

Embora as fontes foquem na modelagem funcional, podem-se identificar os seguintes requisitos implícitos:

RNF01: Segurança: O sistema exige a **validação de senha** para operações como emissão de extrato, saques e depósitos.

RNF02: Integridade: Uma conta não pode ser excluída do sistema após o encerramento, apenas marcada como inativa, para preservar o histórico.

RNF03: Disponibilidade: O sistema deve operar em diferentes interfaces, incluindo **caixas eletrônicos**.

4. DIAGRAMA DE CASOS DE USO

De acordo com a Devmedia, um diagrama de caso de uso (UML) é uma representação visual que define as funcionalidades (o "quê") de um sistema e como os usuários (atores) interagem com ele. Ele foca nos requisitos funcionais, mostrando o comportamento do sistema sob a perspectiva do usuário, sem detalhar como as funções são implementadas.

O diagrama da Figura 1, destaca a fronteira do sistema e a interação entre os atores (**Cliente e Funcionário**):

- **Inclusão (include):** A abertura de conta obriga a realização de um depósito inicial. O encerramento de conta obriga a emissão de saldo.
- **Extensão (extend):** A abertura ou encerramento de conta pode disparar o caso de uso "Manter Cliente" se for necessário atualizar dados ou inativar o registro. Saques ou depósitos são extensões do encerramento de conta, dependendo se o saldo é positivo ou negativo.

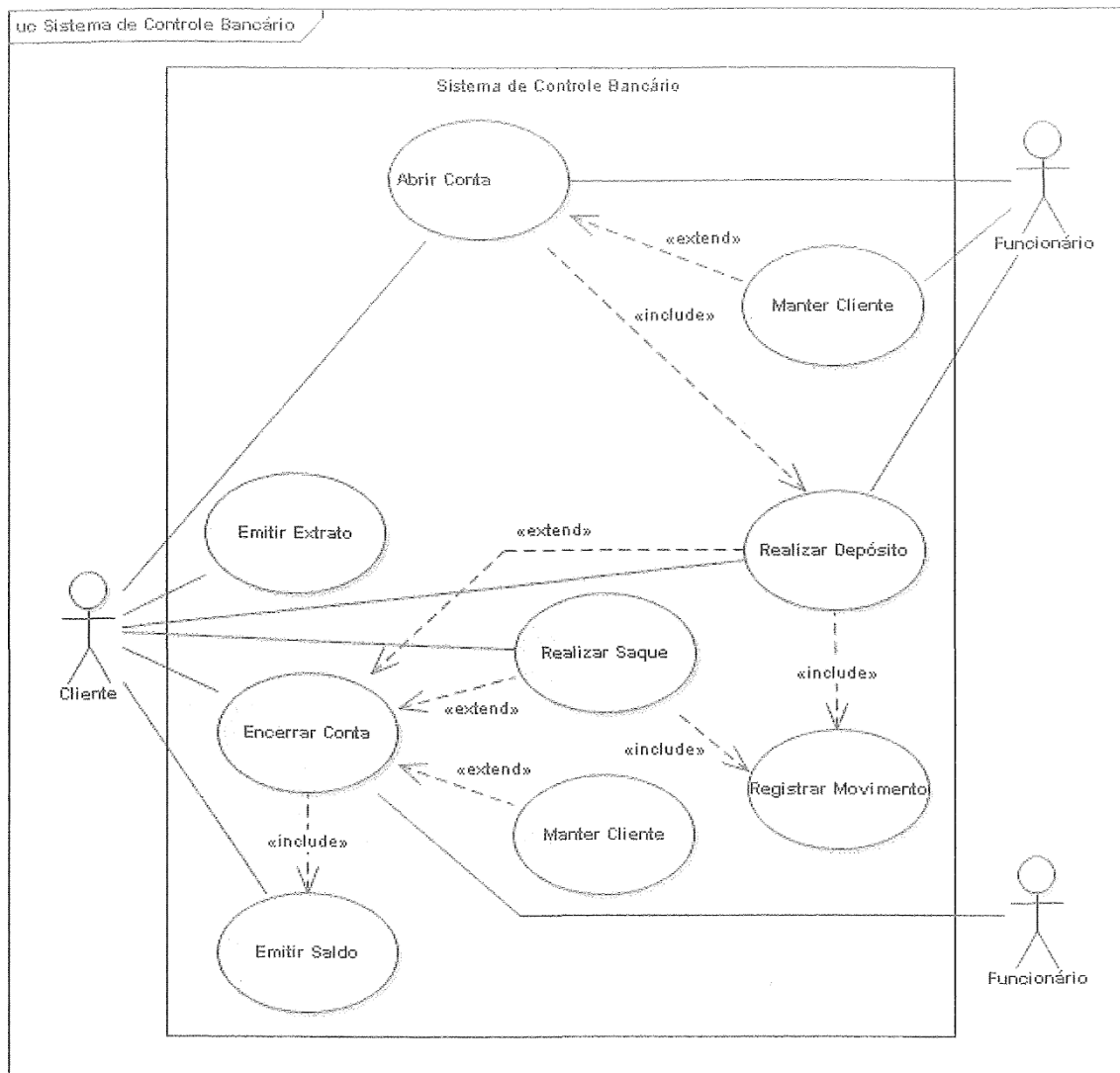


Figura 1: Diagrama de Caso de Uso

5. DIAGRAMAS DE ATIVIDADE

A Microsoft sugere que se use um diagrama de atividade para descrever como várias atividades são coordenadas para fornecer um serviço ou outro resultado final. Diagramas de atividade podem mostrar como os eventos em um caso de uso se relacionam entre si ou como uma coleção de casos de uso coordenam para representar um fluxo de trabalho de negócios.

Emissão de Extrato: Envolve informar a conta, validar a conta, solicitar e validar a senha, escolher o período e apresentar cada movimento encontrado

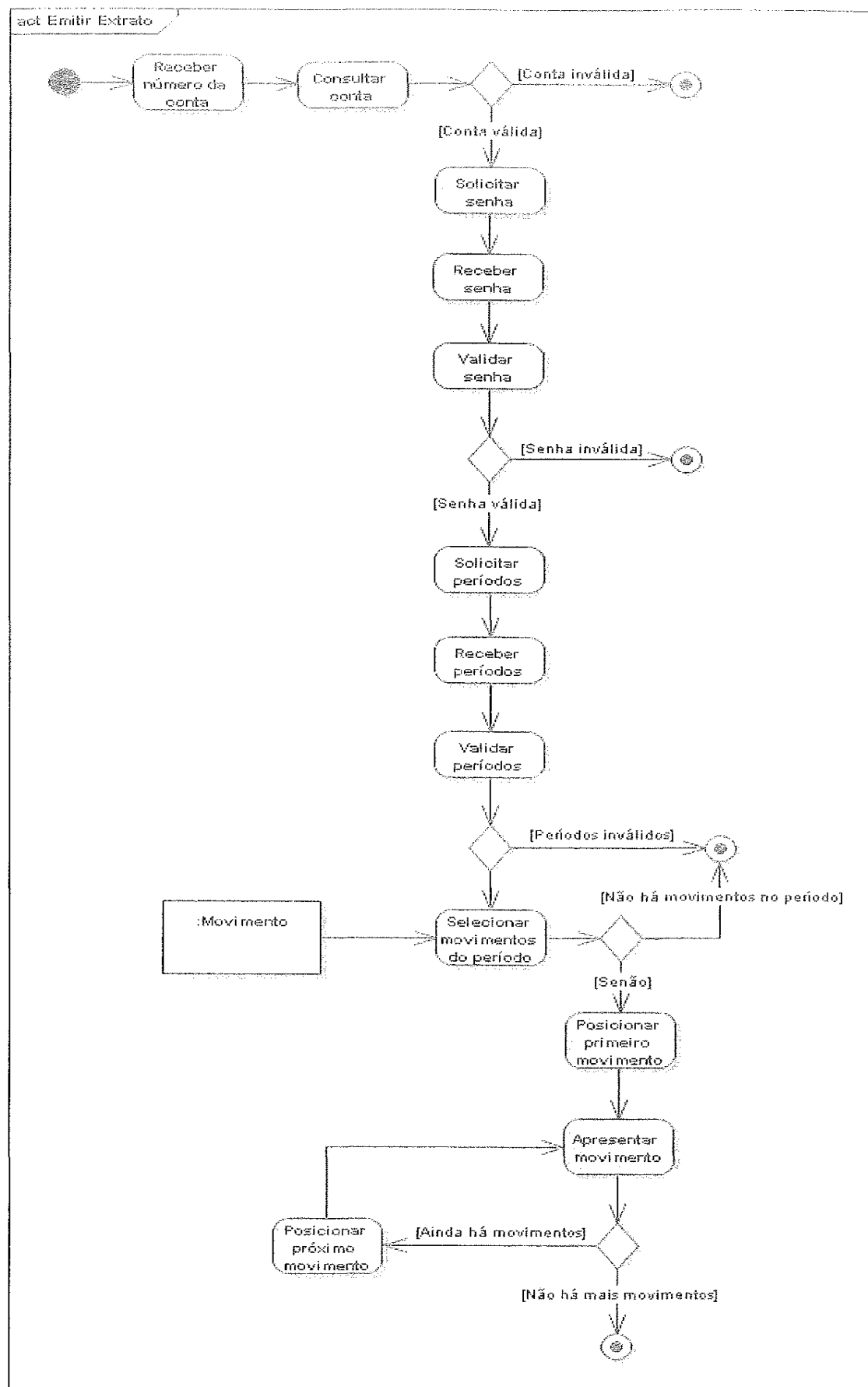


Figura 2: Processo de Emissão de Extrato

Realizar Depósito/Saque: Seguem fluxos similares de identificação e validação, culminando na alteração do saldo e no registro da instância de Movimento.

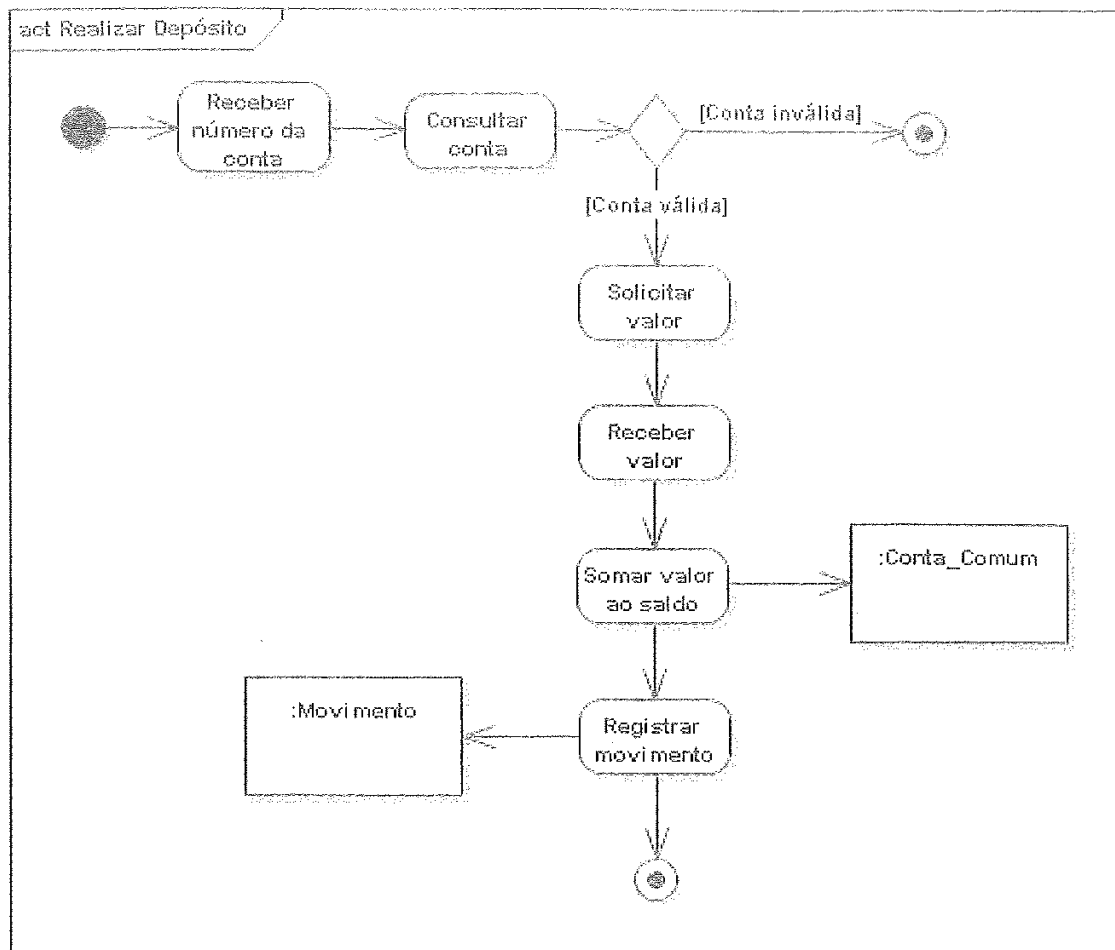


Figura 3: Realizar Depósito

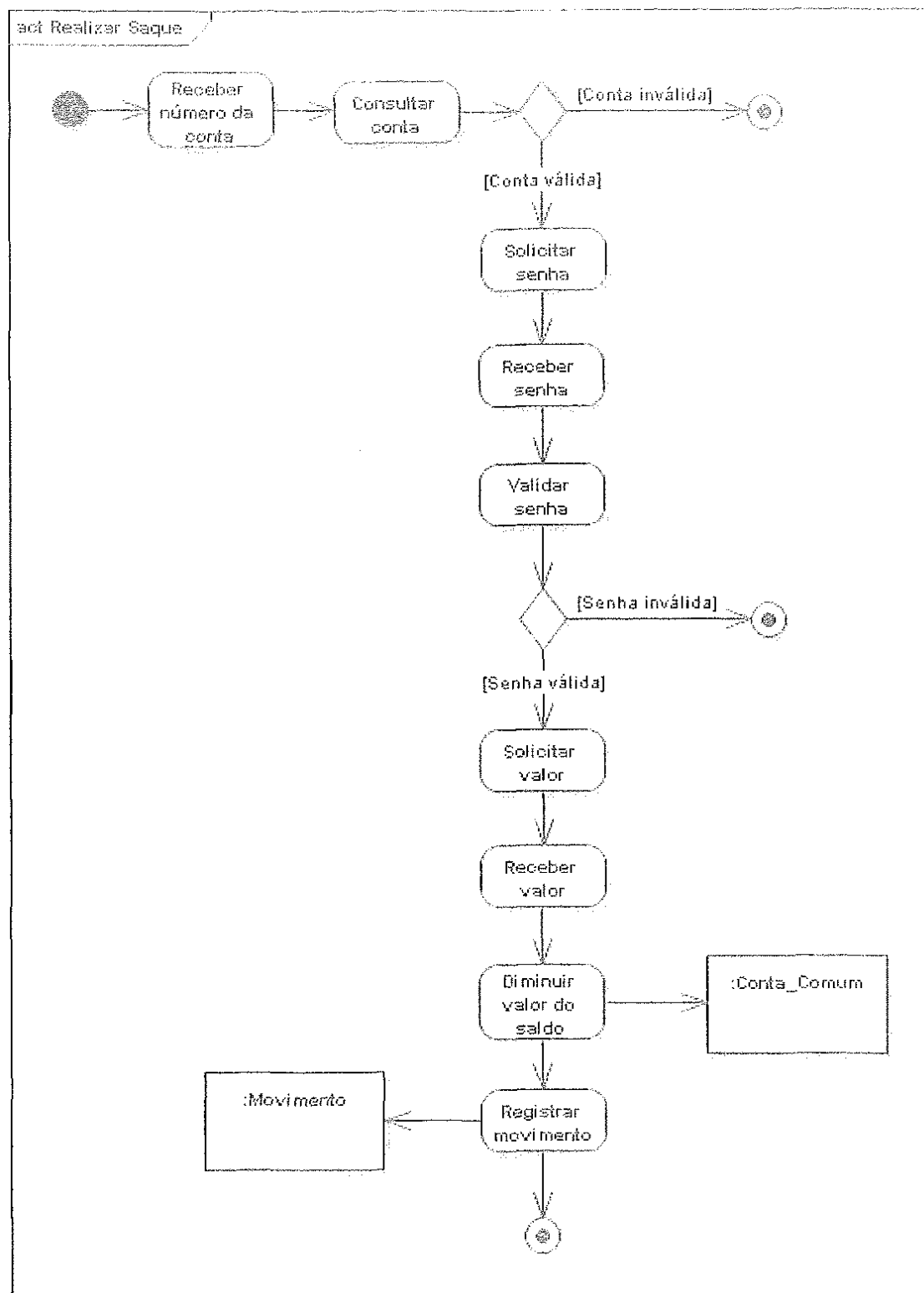


Figura 4: Realizar Saque

Abrir Conta Comum: Processo complexo que envolve consulta de CPF, possível uso da atividade "Manter Cliente", criação do objeto Conta, depósito inicial e emissão do cartão.

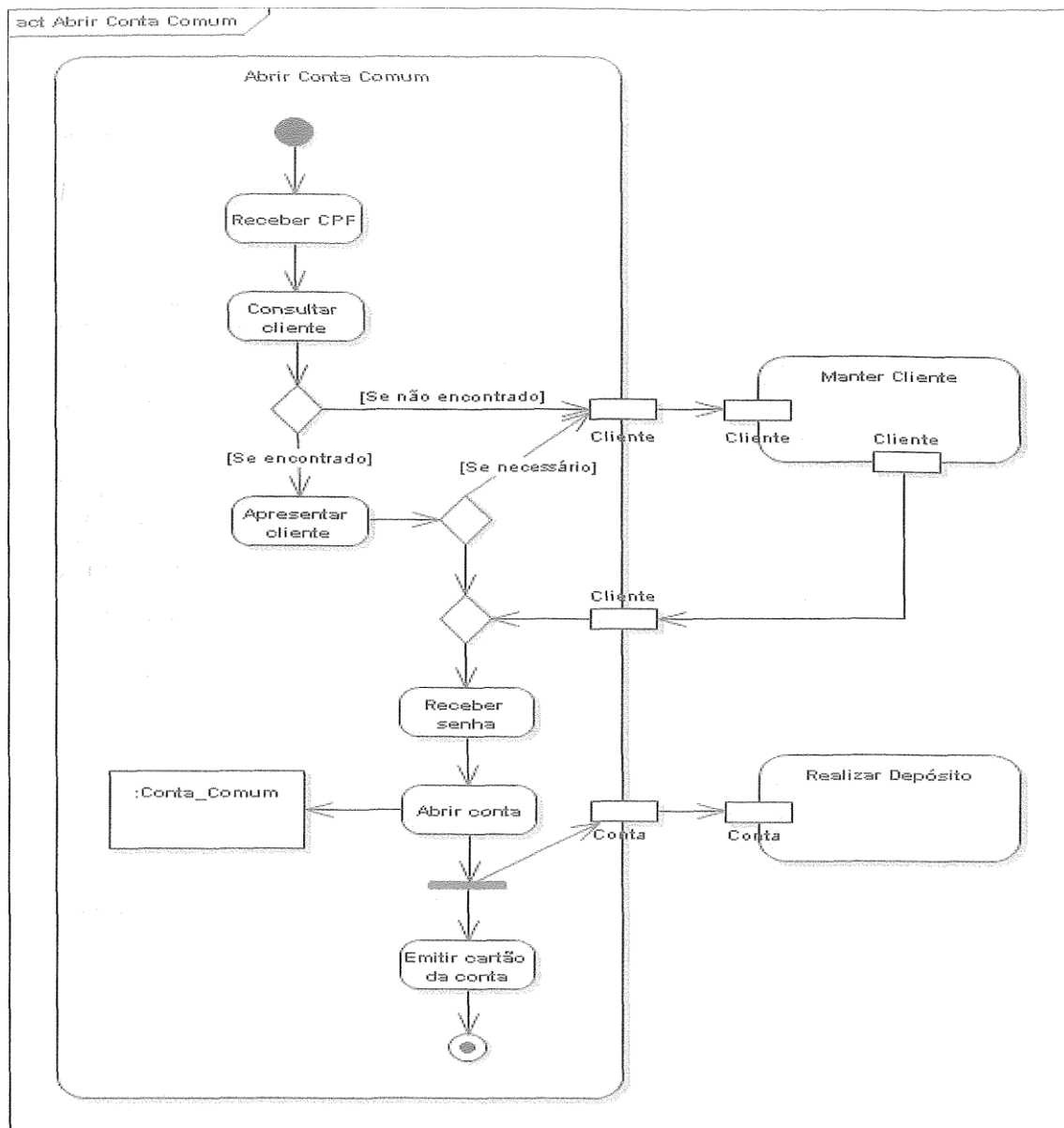


Figura 5: Abrir Conta

Encerramento de Conta: Inicia com a emissão do saldo; se positivo, realiza-se um saque; se negativo, um depósito. Por fim, a situação da conta é alterada para "inativo".

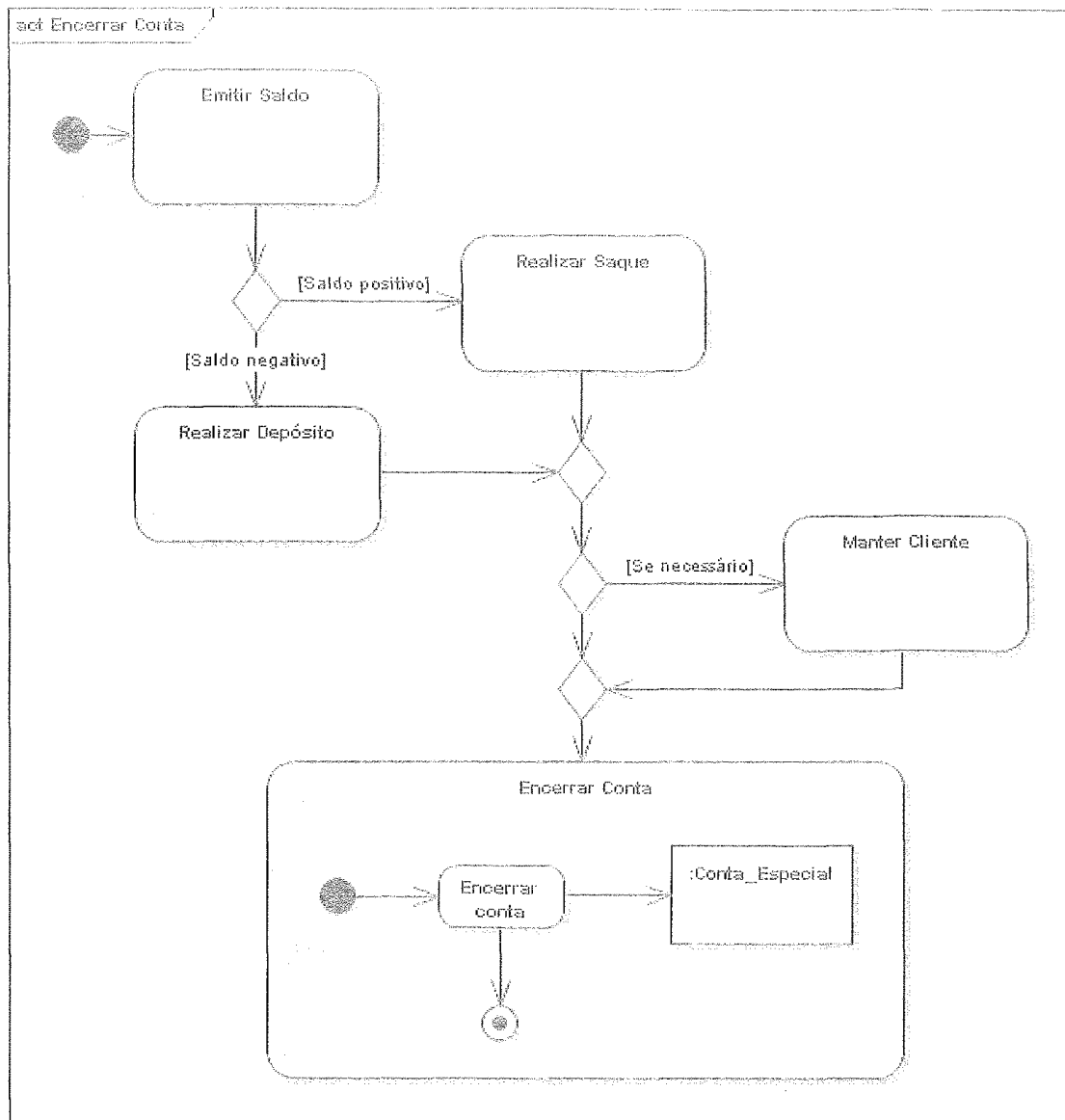


Figura 6: Encerrar Conta

6. DEFINIÇÃO DA IMPLEMENTAÇÃO

O desenvolvimento do Sistema de Controle Bancário será estruturado como uma aplicação web, utilizando a linguagem PHP em seu paradigma procedural e o banco de dados MySQL para a persistência das informações. Para que o sistema seja acessível via navegador, ele será hospedado em um servidor web (como Apache ou Nginx), permitindo que tanto clientes em caixas eletrônicos quanto funcionários em terminais de agência interajam com a interface por meio de requisições HTTP. A escolha do PHP procedural foca na criação de scripts e funções lineares que processam os dados de entrada e

executam consultas diretamente no banco de dados, sem o uso de estruturas de classes ou objetos.

A base de dados no MySQL será composta por tabelas que armazenam os estados das entidades descritas na modelagem. Teremos a tabela de clientes, para guardar dados cadastrais; a tabela de contas, contendo o saldo e o status (ativa/inativa); e a tabela de movimentações, que registrará cada operação de saque ou depósito realizada. Diferente de uma abordagem orientada a objetos, onde se criariam instâncias de classes, a implementação procedural lidará com os registros de forma direta: ao abrir uma conta, o script executará um comando INSERT na tabela de contas, vinculando-a a um registro de cliente e garantindo a inserção do depósito inicial obrigatório.

A lógica de negócio será concentrada em funções PHP responsáveis por validar e processar as operações financeiras. Para o Realizar Saque ou Realizar Depósito, o sistema receberá os dados via formulários web, executará funções de validação de senha e, em seguida, disparará comandos UPDATE para atualizar o saldo na tabela de contas e INSERT para documentar a transação na tabela de movimentações. Para o Encerrar Conta, o script seguirá o fluxo de verificar o saldo atual, emitir os comandos necessários para zerar o montante e, por fim, atualizar o campo de status da conta para "inativo", garantindo que nenhum registro seja fisicamente excluído do banco de dados, preservando o histórico para auditorias.

Para a hospedagem, o ambiente de servidor web garantirá que a aplicação suporte múltiplos acessos simultâneos, gerenciando as sessões de login dos usuários e a segurança das transações. A interface será construída com HTML e CSS, enviando os dados para os scripts PHP no servidor, que processarão as informações e retornarão os resultados ao navegador do usuário, como o saldo atualizado ou a lista de movimentos do período solicitado no extrato. Esta estrutura assegura um sistema leve, eficiente e alinhado aos requisitos de funcionamento de um sistema bancário dinâmico em ambiente de rede.

CONSIDERAÇÕES FINAIS

A elaboração deste projeto de Sistema de Controle Bancário permitiu uma compreensão profunda de como processos operacionais podem ser otimizados através de uma modelagem rigorosa. A utilização dos diagramas de atividades foi determinante para mapear fluxos complexos, como o de **encerramento de conta**, que exige a verificação prévia do saldo e a execução de saques ou depósitos para zerar a conta antes da inativação. Essa visão detalhada assegura que o software não apenas funcione, mas que siga as normas de conformidade e integridade exigidas no setor financeiro. (Informação externa complementada pelas fontes).

Um ponto de destaque na modelagem é a preocupação com a **persistência e integridade dos dados**. Conforme estabelecido, o sistema proíbe a exclusão definitiva de registros de clientes ou contas, optando por alteração de status para "inativo". Essa prática é essencial não apenas para o histórico bancário, mas também para atender a requisitos legais de auditoria e segurança, demonstrando que o modelo proposto está alinhado com as necessidades reais de governança de dados do mercado atual. (Informação externa complementada pelas fontes).

Em conclusão, o desenvolvimento deste modelo acadêmico prepara o futuro analista de sistemas para os desafios reais da indústria de software. A transição da teoria da UML para a aplicação prática em um sistema bancário evidencia que uma boa arquitetura de software é o alicerce para sistemas confiáveis. Ao alinhar as funcionalidades de saque, depósito e manutenção de clientes com uma estrutura de classes e atividades bem definida, este projeto entrega uma base sólida para uma implementação futura eficiente e segura.

REFERÊNCIAS BIBLIOGRÁFICAS

DEVMEDIA. **O que é UML e Diagramas de Caso de Uso: Introdução Prática à UML**. Devmedia, 2012. Disponível em <https://www.devmedia.com.br/o-que-e-uml-e-diagramas-de-caso-de-uso-introducao-pratica-a-uml/23408>. Acesso em 12 de maio de 2026.

GUEDES, Gilleanes T. A. **UML 2: Uma Abordagem Prática**. São Paulo: Novatec

MICROSOFT. **Criar um diagrama de atividade UML**. Microsoft. Disponível em <https://support.microsoft.com/pt-br/topic/criar-um-diagrama-de-atividade-uml-19745dae-2872-4455-a906-13b736f01685>. Acesso em 12 de maio de 2026.

MIRANDA, Luiz. **Requisitos funcionais e não funcionais: o que são, diferenças e exemplos**. Quero Bolso, 2025. Disponível em <https://querobolsa.com.br/revista/requisitos-funcionais-e-nao-funcionais>. Acesso em 12 de maio de 2026.

VALENTE, Marco Tulio. **Engenharia de Software Moderna**. Engsoftmoderna. Disponível em <https://engsoftmoderna.info/cap3.html>. Acesso em 12 de maio de 2026.